

# AICE3001/AICE6002: Mathematics of Machine Learning

## Subsidiary Notes — Lecture 6: Understand Mappings

Adam Prugel-Bennett

October 1, 2026

## 1 Keywords

- Least Squares, Under-constrained Systems, Ill-conditioning

## 2 Main Points

### 2.1 Inverse Problems

- Much of machine learning can be viewed as solving an inverse problem
- We collect data about the world by performing a series of measurements
- Our task is to infer properties of the world from the data
- In linear regression we have  $m$  observations  $(x_k, y_k)$  and want  $p$  unknown weights  $w$  such that  $y_k = x_k^\top w$
- Stacking the feature vectors  $x_k^\top$  as the rows of the *design matrix*  $\mathbf{X}$  (an  $m \times p$  matrix) we can write this as  $y = \mathbf{X} w$
- If  $m = p$  and  $\mathbf{X}$  is invertible then  $w = \mathbf{X}^{-1}y$ , but this is rarely the situation we are in

### 2.2 Least Squares (Over-Constrained Problems)

- We can have contradictory data that our model cannot explain
- This may arise because
  - We have errors in the data
  - Our data contains insufficient information
  - Our model is too simple
- If we have more training data than free parameters ( $m > p$ ) this is likely to occur
- We typically solve this by introducing a loss function we minimise. The classic example is the squared loss

$$L(w) = \sum_{k=1}^m (x_k^\top w - y_k)^2 = \|\mathbf{X} w - y\|^2$$

This was introduced by Gauss to predict the orbit of the asteroid Ceres

- Setting the gradient to zero (see Exercise 1) gives

$$w = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top y = \mathbf{X}^+ y$$

where  $\mathbf{X}^+ = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$  is the pseudo-inverse

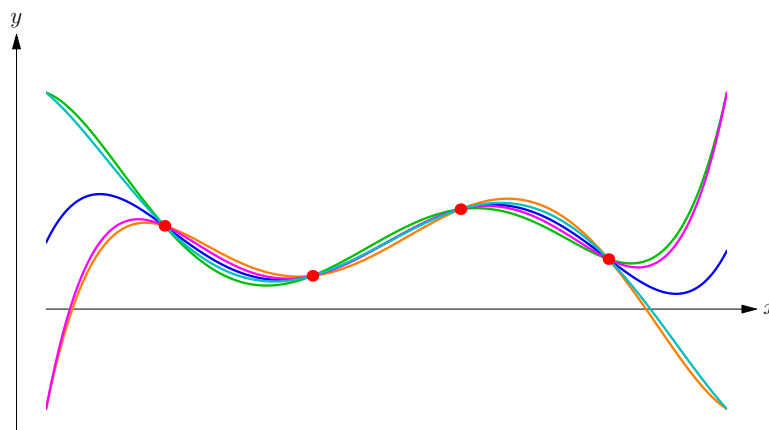
- This requires  $\mathbf{X}^\top \mathbf{X}$  to be invertible, which is the case when the columns of  $\mathbf{X}$  are linearly independent

## 2.3 Under-Constrained Problems

- We can also be in a situation when many models (learning machines) explain the data
- This will typically happen when we have more free parameters than data ( $m < p$ ), or when we are unlucky and the training examples don't span the full space
- In linear regression  $\mathbf{X}^T \mathbf{X}$  is then singular (see Exercise 3), so the pseudo-inverse formula fails. The problem is said to be *ill-posed*
- As an example, consider fitting a polynomial of degree 5,  $y = w_0 + w_1 x + \dots + w_5 x^5$  (6 parameters), to 4 data points  $(x_i, y_i)$ . If  $p_0(x)$  is the cubic through the four points then

$$p(x) = p_0(x) + (a + bx)(x - x_1)(x - x_2)(x - x_3)(x - x_4)$$

passes through all four points for *every* choice of  $a$  and  $b$  (Exercise 4). We have 6 unknowns and 4 equations, leaving 2 free directions. The figure shows five members of this family. They agree near the data but make wildly different predictions away from it



- Here we have to choose a particular model. To do this requires (implicitly or explicitly) making additional assumptions
- Ockham's razor suggests choosing the "simplest" solution, although what counts as simplest is a choice we make
- For linear regression, adding an  $L_2$  regulariser  $\eta \|\mathbf{w}\|^2$  to the loss makes the solution unique. We will see exactly how this works when we meet the singular value decomposition
- For high-dimensional inputs we can be over-constrained in some directions and under-constrained in others

## 2.4 Ill-Conditioning

- Even when we are not under-constrained our inverse can be very sensitive to the data
- That is, small errors can be strongly magnified
- A matrix that is close to singular is ill-conditioned: some of its eigenvalues are tiny compared with the largest, so it squashes space towards a lower-dimensional subspace. Inverting it stretches those directions back out, magnifying any noise in them
- A worked example: take two data points with two nearly identical features

$$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1.001 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 2 \\ 2.001 + \delta \end{pmatrix}$$

Solving  $\mathbf{X} \mathbf{w} = \mathbf{y}$ , subtracting the first equation from the second gives  $0.001 w_2 = 0.001 + \delta$ , so

$$\mathbf{w} = \begin{pmatrix} 1 - 1000 \delta \\ 1 + 1000 \delta \end{pmatrix}$$

- A change  $\delta$  in the data produces a change  $1000 \delta$  in the weights. With  $\delta = 0.001$  (a 0.05% change in  $y_2$ ) the weights jump from  $(1, 1)$  to  $(0, 2)$  and the prediction for a new input  $\mathbf{x} = (1, 0)^\top$  goes from 1 to 0
- The weights always move along  $(-1, 1)^\top$ . This is (very nearly) the eigenvector of  $\mathbf{X}$  with the small eigenvalue ( $\approx 0.0005$ , against  $\approx 2$  for the other one)
- The data hardly constrain the weights in this direction: the two features are so alike that the data cannot tell us how to share the weight between them
- Common causes of ill-conditioning in machine learning
  - Roughly as many data points as parameters
  - Nearly collinear features (e.g. floor area and number of rooms)
  - Polynomial features  $x, x^2, \dots, x^9$  for  $x \in [0, 1]$ , which all look alike (see the experiments)
  - Features on very different scales (easily fixed by standardising each feature to zero mean and unit variance)
- Ill-conditioning leads to high variance in the bias-variance sense and hence poor generalisation
- Regularisers are one way of improving the conditioning

## 2.5 Least Squares in Practice

- The formula  $\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$  is for understanding, not for computing
- Use a least squares solver such as `np.linalg.lstsq(X, y)`, which works with  $\mathbf{X}$  directly
- Forming  $\mathbf{X}^\top \mathbf{X}$  squares the ratio of the largest to smallest eigenvalues (the *condition number*, defined properly in the next lecture). If that ratio is  $10^8$  for  $\mathbf{X}$  it becomes  $10^{16}$  for  $\mathbf{X}^\top \mathbf{X}$ , beyond the roughly 16 significant figures of a double-precision number
- When  $m < p$ , `lstsq` returns the solution with the smallest  $\|\mathbf{w}\|$ , which is one version of the "simplest" solution. We will see why when we meet the singular value decomposition

## 2.6 Three Regimes

| Regime                      | Exact fit?  | Unique?       | Method                                             |
|-----------------------------|-------------|---------------|----------------------------------------------------|
| $m > p$ (over-constrained)  | usually not | yes (usually) | minimise $\ \mathbf{X}\mathbf{w} - \mathbf{y}\ ^2$ |
| $m = p$                     | yes         | yes (usually) | $\mathbf{w} = \mathbf{X}^{-1} \mathbf{y}$          |
| $m < p$ (under-constrained) | yes, many   | no            | choose the "simplest"                              |

- "Usually" means provided the columns of  $\mathbf{X}$  are linearly independent
- Real problems mix these: over-constrained in some directions and (nearly) under-constrained in others. The nearly under-constrained directions are what make a problem ill-conditioned

### 3 Exercises

1. Derive the formula  $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$  for the weight vector in linear regression by minimising  $\|\mathbf{X} \mathbf{w} - \mathbf{y}\|^2$
2. By writing  $\mathbf{w}^T \mathbf{M} \mathbf{w} = \sum_{i,j} w_i M_{ij} w_j$  and differentiating with respect to  $w_k$ , show that  $\nabla \mathbf{w}^T \mathbf{M} \mathbf{w} = (\mathbf{M} + \mathbf{M}^T) \mathbf{w}$
3. Show that if  $\mathbf{X}$  is an  $m \times p$  matrix with  $m < p$  then  $\mathbf{X}^T \mathbf{X}$  is singular. (Hint: find a non-zero vector  $\mathbf{v}$  with  $\mathbf{X} \mathbf{v} = \mathbf{0}$ )
4. For the polynomial example in the notes
  - (a) Show that  $p(x)$  passes through all four data points for any  $a$  and  $b$ , and that it is a polynomial of degree at most 5
  - (b) (Harder) Show that every polynomial of degree at most 5 through the four points has this form
5. For the worked example with  $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1.001 \end{pmatrix}$ 
  - (a) Verify the solution  $\mathbf{w} = (1 - 1000\delta, 1 + 1000\delta)^T$
  - (b) Find the eigenvalues of  $\mathbf{X}$  to 4 significant figures and their ratio
  - (c) What happens to the weights if instead the two features were identical,  $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ ?

### 4 Experiments

#### 4.1 Ill-Conditioning and Under-Constrained Systems

- Using python (with numpy), try the following and check you understand the output

```
import numpy as np
```

```
# 1. The worked example: a tiny change in y, a big change in w
X = np.array([[1, 1], [1, 1.001]])
for y2 in [2.001, 2.002, 2.0005]:
    print(y2, np.linalg.solve(X, [2, y2]))
print(np.linalg.eigh(X)) # eigenvalues and eigenvectors (X is symmetric)
```

```
# 2. Polynomial features are ill-conditioned
x = np.linspace(0, 1, 50)
for degree in [1, 3, 5, 9]:
    Xp = np.vander(x, degree + 1, increasing=True) # columns 1, x, x^2, ...
    print(degree, np.linalg.cond(Xp), np.linalg.cond(Xp.T @ Xp))
```

```
# 3. An under-constrained system: 3 equations, 6 unknowns
rng = np.random.default_rng(1)
X = rng.normal(size=(3, 6))
y = rng.normal(size=3)
w, *_ = np.linalg.lstsq(X, y)
print(np.allclose(X @ w, y)) # fits the data exactly
print(np.linalg.eigvalsh(X.T @ X)) # three (numerically) zero eigenvalues
v = np.linalg.svd(X)[2][-1] # a direction with X v = 0
print(np.allclose(X @ (w + 5 * v), y)) # ...so this also fits exactly
print(np.linalg.norm(w), np.linalg.norm(w + 5 * v)) # but a larger norm
```

- In part 2, `np.linalg.cond` computes the condition number (covered next lecture). Notice that the condition number of  $\mathbf{X}^T \mathbf{X}$  is the square of that of  $\mathbf{X}$ , reaching about  $10^{13}$  for degree 9
- In part 3, try adding other multiples of  $\mathbf{v}$ . Why can the norm never be smaller than that of  $\mathbf{w}$ ?

## 5 Answers

1. Using  $\|\mathbf{a}\|^2 = \mathbf{a}^T \mathbf{a}$  and  $\mathbf{w}^T \mathbf{X}^T \mathbf{y} = \mathbf{y}^T \mathbf{X} \mathbf{w}$  (a number equals its own transpose)

$$\|\mathbf{X} \mathbf{w} - \mathbf{y}\|^2 = \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - 2 \mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{y}^T \mathbf{y}$$

As  $\mathbf{X}^T \mathbf{X}$  is symmetric, Exercise 2 gives the gradient  $2(\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y})$ . Setting this to zero gives  $\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$  and, if  $\mathbf{X}^T \mathbf{X}$  is invertible, the result

$$2. \quad \frac{\partial}{\partial w_k} \sum_{i,j} w_i M_{ij} w_j = \sum_j M_{kj} w_j + \sum_i w_i M_{ik} = (\mathbf{M} \mathbf{w})_k + (\mathbf{M}^T \mathbf{w})_k$$

3. The  $p$  columns of  $\mathbf{X}$  are vectors in  $\mathbb{R}^m$  with  $m < p$ , so they are linearly dependent: some non-zero  $\mathbf{v}$  has  $\mathbf{X} \mathbf{v} = \mathbf{0}$ . Then  $\mathbf{X}^T \mathbf{X} \mathbf{v} = \mathbf{0}$ , so  $\mathbf{v}$  is an eigenvector of  $\mathbf{X}^T \mathbf{X}$  with eigenvalue 0 and  $\mathbf{X}^T \mathbf{X}$  has no inverse. Any multiple of  $\mathbf{v}$  can be added to a solution without changing the fit

4. Polynomial example

- (a) At each data point one factor  $(x - x_i)$  vanishes, so  $p(x_i) = p_0(x_i) = y_i$ . The degree is at most  $1 + 4 = 5$
- (b) If  $r(x)$  is another such polynomial then  $r(x) - p_0(x)$  has degree at most 5 and is zero at  $x_1, \dots, x_4$ , so it is divisible by  $(x - x_1) \cdots (x - x_4)$ . The quotient has degree at most 1, i.e. it is  $a + bx$  for some  $a$  and  $b$

5. Worked example

- (a) Substitute back:  $(1 - 1000\delta) + (1 + 1000\delta) = 2$  and  $(1 - 1000\delta) + 1.001(1 + 1000\delta) = 2.001 + \delta$
- (b) The eigenvalues satisfy  $\lambda^2 - 2.001\lambda + 0.001 = 0$ , giving  $\lambda \approx 2.001$  and  $\lambda \approx 0.0004999$ , a ratio of about 4000
- (c)  $\mathbf{X}$  is singular (eigenvalues 2 and 0). If  $y_1 = y_2$  there are infinitely many solutions,  $w_1 + w_2 = y_1$ ; otherwise there is no exact solution. The ill-conditioned problem has become under-constrained in the  $(-1, 1)^T$  direction